Problema del Cambio (Greedy)

Se tiene un sistema monetario (ejemplo, el nuestro). Se quiere dar "cambio" de una determinada cantidad de dinero. Implementar un algoritmo que devuelva el cambio pedido, usando la mínima cantidad de monedas/billetes.

Estrategia: la mayor cantidad de la moneda más grande

En general el algoritmo que resuelve este problema sale de forma bastante intuitiva: agarramos la moneda inmediatamente menor (o igual) al cambio que estamos tratando de generar, y ver cuántas veces podemos utilizar dicho cambio.

Es decir:

```
def cambio(n, monedas): # monedas esta ordenado de menor a mayor, por supuesto
    resultado = {} # podríamos tener un arreglo/lista si quisiéramos
    restante = n
    for moneda in monedas[::-1]:
        resultado[moneda] = restante // moneda
        resultado %= moneda
    return resultado
```

Esto va a tener complejidad $\mathcal{O}(C)$, siendo C la cantidad de monedas en mi sistema. Lo cual es bueno, porque en la mayoría de los casos reales, podemos considerar a esta cantidad constante.

¿El algoritmo es greedy?

Respuesta corta: si. Mi regla greedy: uso la mayor cantidad de monedas de la moneda más grande que pueda (la inmediatamente inferior).

Incluso, se puede segregar a simplemente utilizar la moneda más grande una vez (lo de ver cuántas veces se puede usar, es nada más una optimización para la complejidad).

¿Cuál es el óptimo local al que llegamos? A que, para con esta moneda, nos quedamos con el mínimo posible de cambio restante luego de esta etapa. Sea que lo consideremos como agarrar una unidad de la moneda o todas las posibles, para esta etapa usamos para que nos quede el mínimo posible de cambio restante, buscando que eso haga que nos haga tener la menor cantidad de cambio posible.

¿Es óptimo?

Ni. Depende del sistema monetario. En sistemas monetarios modernos es óptimo. En sistemas monetarios como el británico anterior o en el sistema de Harry Potter, no era óptimo.

Ponemos de ejemplo, teniendo las monedas `[1, 5, 6, 9]` y queriendo cambio para `n = 11`. En este caso, el óptimo es `[5, 6]`, pero nuestro algoritmo devolverá `[1, 1, 9]`.

¿Y cuándo sé cuándo es óptima? Hay un paper al respecto. Dejamos acá una explicación detallada del algoritmo que permite determinarlo.